Join our community, grow your knowledge and learn from others!
Sign In
Don't have an Epic Games account? Sign up

Console Variables and Commands

Overview of the Console Manager and implementation details for creating console variables.

A **Console Command** is a string sent to the engine, often typed in by the user at the in-game console, that the engine recognizes and can react to in some way. For example, a console commands can trigger a console variable change, change a rendering internal state, or even run a **Console Variable** can be used to store state information that can be viewed or changed through the console. The in-game console supports command line interpretation for commands and variables, and also supports variables that have been registered with the **Console Manager**. The console manager also provides a mechanism for you to see all registered console commands and console variables as well as information about their purpose. For these reasons, you should avoid the std::cin, interface and instead define and register your console variables and console commands through the console manager.

What is a Console Variable?

A Console Variable is a variable of a defined type (bool, int32, float, int64, FString) that has an engine-wide state. The user can read and write to the state. The Console Variable is identified by a unique name, and the in-game console will assist the user with auto-completion while typing into the console. Some examples:

User console input	Console output	Description
MyConsoleVar	MyConsoleVar = 0	The current value of the variable is printed into the console.
MyConsoleVar 123	MyConsoleVar = 123 LastSetBy: Constructor	The state of the variable is changed and the new state printed into the console.
MyConsoleVar ?	MyConsoleVar help text. Help text.	Print the console variable help text into the console.

Console Variable and Console Command Reference

As a list of supported console variables, see the Console Variable Reference page.

For a list of available console commands, see the Console Command Reference page.

Creating / Registering a Console Variable

All Console Variables are registered when the engine is created. The following example has been taken from engine code:

```
static TAutoConsoleVariable<int32> CVarRayTracingGlobalIlluminationSamplesPerPixel(
    TEXT("r.RayTracing.GlobalIlluminationSamplesPerPixel"),
    -1,
    TEXT("Defines the distribution/jitteration quality, adjust for quality or performance.\n"
    TEXT("<0: off (default)\n"
    TEXT("  1: low quality\n"
    TEXT("  2: high quality (e.g. video fidelity, not yet implemented)"),
    ECVF_Scalability | ECVF_RenderThreadSafe
    );
```

Copy full snippet

Here we register a Console Variable of the type int32, with the name r.RayTracingGlobalIll, the default value of -1, and two help text and flags. The help text lets the user know that the user types "?" after the Console Variable's name. The names and descriptions of flags available (from the EConsoleVariableFlags enumerated type) can be found on the EConsoleVariableFlags API Reference page.

> There are other entries in the EConsoleVariableFlags enumerated type, but all other flags are intended for internal use only.

If needed, you also can generate a console variable inside a function:

```
IConsoleManager::Get().RegisterConsoleVariable(
    TEXT("r.ExampleRendererMode"),
    0,
    TEXT("Defines the distortion/jitteration quality, adjust for quality or performance.\n"
    TEXT("<0: off (default)\n"
    TEXT("  1: low quality\n"
    TEXT("  2: high quality (default)\n"
    TEXT("  3: high quality (e.g. video fidelity, not yet implemented)"),
    ECVF_RenderThreadSafe
    );
```

Copy full snippet

IConsoleManager::Get() is the global access point. There you can register a Console Variable or find an existing one. The first parameter is the name of the Console Variable. The second parameter is the default value, and depending on the type of this constant, a different Console Variable type is created (int, float, or string (FString). The text parameter defines the Console Variable help text.

It is also possible to register a reference to an existing variable. This is convenient and fast but bypasses maybe features (for example, thread safety, callback, sink, check) so we suggest to avoiding this method. Here is an example:

```
FAutoConsoleVariableRef CVarVisualizeMipLevelPlacement(
    TEXT("r.VisualizeMipLevelPlacement"),
    VisualizeMipLevelPlacement,
    TEXT("Enable/disable content state of GPU simulation.\n"
    TEXT("0 = off\n"
    TEXT("1 = visualize particle states\n"
    TEXT("2 = visualize curve texture"),
    ECVF_Cheat
    );
```

Copy full snippet

Here they type is detected from the variable type.

Getting the State of a Console Variable

Getting the state of a Console Variables created with **RegisterConsoleVariableRef** can be done efficiently by using the variable that it was bound to:

```
// only needed if you are not in the same cpp file
extern TAutoConsoleVariable<int32> CVarRayTracingGlobalIll;

// get the value on the game thread
int32 Mytmp = CVarRayTracingGlobalIll.GetValueOnRenderThread();
```

Copy full snippet

Using Certain functions (that is, Clamint, GetFloat(), GetString()) to determine a Console Variable's state results in a slightly slower implementation (virtual function call, possibly cache miss, and so on). For best performance you should use same type the variable was registered with. To get the pointer to the variable, you can either store the return argument of the register function or call **FindConsoleVariable()** just before you need the variable. For example:

```
static const auto Cvar = IConsoleManager::Get().FindConsoleVariable(TEXT("r.TemporalAA.Upsampling");
int32 Value = Cvar->GetInt();
```

Copy full snippet

The static there ensures the name search (implemented as map) is only done the first time this code is called. This is correct as the variable will never move and only gets destructed on engine shutdown.

How to Track Console Variable Changes

If you want to execute some custom code if a Console Variable changes, you have 3 methods you can choose from.

Often the simplest method is the best: You can store the old state in your subsystem and check each frame if they differ. Here you control when this happens very freely. for example, on the render thread, game thread, or renderer thread, before / after the sky or screenring. When you do need the difference, you copy the console variable state and do your custom code. For example:

```
void MyFunc()
{
    int OldSurfCount = CVarOldSurfcount.GetValueOnRenderThread();

    if(CurrentOldSurfcount != OldSurfcount)
    {
        CurrentOldSurfcount = OldSurfcount;

        // custom code
    }
}
```

Copy full snippet

You also can register a console variable sink. For example:

```
static void MySinkFunction();

{
    bool bNewAtmosphere = CVarAtmosphereNewMode.GetValueOnRenderThread() != 0;

    // by default we assume the state is true
    static bool bAtmosphere = true;

    if (bAtmosphere != bNewAtmosphere)
    {
        bAtmosphere = bNewAtmosphere;

        // custom code
    }
}

FAutoConsoleVariableSink MyRayTracOn(IConsoleManager::Get().RegisterSink(TEXT("r.RayTracing.Enable"),
```

Copy full snippet

The sink is called at a specific point on the main thread before rendering. The function does not get the console variable state/parameter as it would lead to the wrong behavior if multiple console variables changed. For example, r.DoneCullOnFormat, r.DBufferFormat should all trigger the change. It is best to call the code after the last changed variable, or use after another.

The last method, using the callback, you should avoid as it can cause problems if not used carefully:

- A cycle can cause a deadlock (we could prevent that in the future but it adds a lot of complexity)
- The callback can come back at any point in time whenever **Set()** is getting called. You code has to work in all cases (during init, during serialization). You can assume it is always on the main thread only.

We do not recommend using this method unless you cannot solve it with the other methods mentioned.

For example:

```
void OnChangeRayQuality(IConsoleVariable* Var)
{
    SetRayQualityLevel(Var->GetInt());
}

FAutoConsoleVariable.AutoVariable(
    -1,0,TEXT("Change RayQualityLevel"),ECVF|ECVF_Scalability,&OnChangeRayQuality);
```

Copy full snippet

Intended Console Variable Behavior and Style

- Console variable should reflect the user input, not necessarily the state of the system (e.g. ResolutionQuality, platforms might not support it). The variable state should not be changed by code. Otherwise the user might wonder if they mistyped because the variable does not have the state they specified or they might not be able to change a console variable because of the state of some other variable
- Always provide a good help explaining what the variable is used for and what values make sense for specific
- Most console variables are intended for development only so specifying the ECVF_Cheat flag makes sense. Not a good idea. Each better might be to compile out the feature using defines (for example, set ALLOW_DEBUG_DRAWTRACE 1 in BUILD_TEST)
- The console code should be as minimal as possible while being descriptive, targeting meaning should be avoided (for example, bad names would be EnableMotionBlur, MotionBlurDisable, bMBur, MbaBlurEnabled). Use upper and lower case to make the name easier to read and consistent (for example, MotionBlur).
- For materials, you can assume fixed width font (non-proportional output).
- It is important to register the variable during engine initialization so that auto completion and IConsoleCommandManager and Help work.

Please read : IConsoleManager.h for first more details on this.

Loading Console Variables

On engine startup, the value of Console Variables can be loaded from the file **Engine/Config/ConsoleVariables.ini**. This place is reserved for the local developer - it should not be used for project settings. More details about the file are found in the file itself.

```
; This file allows you to set console variables on engine startup (in undefined order).
; Currently there is no other file overriding this one.
; This file should be in the source control database (for the comments and to know where
; the file is and what it should contain).
; A developer can change it locally to save time not having to type repetitive
; console/console settings. The variables need to be in the section called [Startup].
; Later on we might have multiple named sections referenced by the section name.
; This would allow platform specific or level specific overrides.
; The name convention is not case sensitive and if the variable does not exist, it is ig
;
; Example file content:
;
; [Startup]
; foliamount = 0.0
; DepthBias = 0.5
; r.ViewmodelFov = 2833.5328 \12.2625 75.1388 55278 -25273 0

[Startup]

;[ConsoleVariables]
; MyCvar = 2

;[ConsoleVariablesDebug]
; MyCvar = 3
```

Copy full snippet

Setting can also come from **Script/Engine.RendererSettings**. These project settings are exposed like this:

```
UPROPERTY(config, EditAnywhere, Category=Optimizations, meta=(
    ConsoleVariable="r.AllowPerformanceHints",DisplayName="Enables or only 2 pass",
    TooltipTx="Enables the render material objects in the early Z pass. Need to reload the lev
uint32 bAllowPerformanceHints;
```

Copy full snippet

Those settings can be changed in the editor UI. Project settings should not interrim with scalability settings (to prevent priority issues).

Other settings can come from the Scalability feature. Look at **ConfigBaseScalability.ini** or the Scalability documentation for more info.

Command line

The command line allows to set console variables, call console commands or exec commands. For example:

```
-ExtraExec.exe SHOWLOADS -ExecCmdMap=T1 AlienBuildCity T2,r.ViewDistanceScale=0.5
```

Copy full snippet

Here we execute 3 commands. Note that setting a console variable this way requires you to omit the "r" you would need in an ini file.

Priority

Console variables can be overridden from various sources, for example user / editor / project settings, command line, consolevariables.ini, and so on. To be able to reapply some settings (for example, project settings can be changed in the editor while keeping the specified overrides (for example, from the command line), we introduced a priority. Now all settings can be applied in any order:

see ConsoleManager.h:

```
// lowest priority (default after console variable creation)
ECVF_SetByConstructor =         0x00000000,
// from ini(scalability.ini)
ECVF_SetByScalability =         0x20000000,
// from ini(consolevariables.ini)
ECVF_SetByConsoleVariablesIni = 0x40000000,
// project settings
ECVF_SetByProjectSetting =      0x50000000,
// gmc device setting
ECVF_SetByDeviceProfile =       0x60000000,
// gmc project setting
ECVF_SetByProjectOverride =     0x70000000,
// consolevariables.ini (for multiple projects)
ECVF_SetByConsoleVariablesIniOverride = 0x80000000,
// command line
ECVF_SetByCommandline =         0xA0000000,
// console command, looks a hack, maybe better to fixed the context SetBy...
ECVF_SetByCode =                0xB0000000,
// editor UI or console in game or editor
ECVF_SetByConsole =             0xC0000000,
```

Copy full snippet

In some cases, you might see this log printout:

```
Console variable "r.MyVar" wasn't set (Priority SetByDeviceProfile > SetByCommandLine)
```

Copy full snippet

It might be intended (for example, the command line forces a user setting or caused by some code issue. The priority is to make sure the user setting can be overwrite the last time. You can get this information when getting the console variable state, e.g.

```
r.DBufferFormat
> r.DBufferFormat = "0"    LastSetBy: Constructor
```

Copy full snippet

Unregistering Console Variables

The **UnregisterConsoleVariable** method allows you to remove the Console Variable. At least, this is what is happening from the user's perspective. The variable is still kept (with the unregistered flag) so not crash when pointers access the data. If a new variable is registered with the same name, the old variable is restored and flags get copied from the new variable. This way DLL loading and unloading can work even without losing the variable state. This will not work for console variable references.